Terms

Ten words that carry the whole method. Everything else is built from these.

entropy
The randomness from the dice — the raw material the 24 words are built from.

2-of-3
Any two of three keys can spend. **One alone cannot.**

PSBT
The unsigned payment file, carried on the USB. The seed never leaves the offline computer.

BIP39
The 2,048-word list the 24 words come from.

signature
The project confirming the file. **“Good signature”** — or the file is deleted.

airgap
Tails booted with networking **Disabled**. The computer forgets the session on shutdown.

m/48'/0'/0'/2'
The wallet’s path. **Confirmed on screen**, never typed from memory. Not a secret.

descriptor / xpub
The wallet map / a watch key. Both **watch**, neither **spends**.

hash · fingerprint
The file’s fingerprint must match the project’s published one — **every character**.

zero-trust
Nobody’s word is taken. **Two tools must agree.**

Checklist

Everything on the table before the first boot. Nothing is ordered mid-build.

ON THE TABLE

The dice table

- **Five casino dice** with readable faces
- **Pencil** and the recording grid
- A **private room** — no camera

THE COMPUTER

One machine, two boots

- **Boot 1:** stays online, watch-only, never holds a seed
- **Boot 2:** Tails USB, networking off
- **Data USB:** carries only checked files

FILES

Checked before anything goes offline

- **Tails 7.13** · **Sparrow 2.5.5** (exact) · **Electrum 4.8.2**
- **Ian Coleman** page (offline only) · **dice converter** (in this kit)

METAL

Three plates, three places

- **Three titanium plates** — one seed on each
- **StampSeed** kit holds 1 (buy 2 more) · **Cryptotag** holds 2 (buy 1 more) — don’t mix methods
- Three places, **far enough apart** that one disaster can’t take all three

> **Nothing is bought mid-build.** If a plate or USB is missing when its sheet arrives, stop and get it first.

Check the files

Every file gets checked before the computer goes offline. A file that fails is deleted. No exceptions — a compromised file compromises the funds.

What is required

- The downloads from the checklist: Tails 7.13, Sparrow 2.5.5, Electrum 4.8.2, the Ian Coleman page, and the dice converter from this kit.

1. Download only from the real sites.

Sparrow must be 2.5.5 exactly. A newer download will not match — that is a stop, not an upgrade. The dice converter is the one file this site provides; its fingerprint is in VERIFY.md.

2. Compare the fingerprint. Every character.

Windows: PowerShell → Get-FileHash. Mac: Terminal → shasum -a 256. Paste the result next to the fingerprint published on the project's own site.

3. Check the signature. Want: “Good signature”.

Import the signing key, confirm its fingerprint, then verify. The “not certified” warning is normal on first import. What matters: the line says Good signature and the fingerprint matches.
Sparrow signing key: D4D0D3202FC06849A257B38DE94618334C674B40.
Electrum (ThomasV): 6694D8DE7BE8EE5631BED9502BD5824B7F9470E6.

4. Run the kit check.

Put the downloads next to verify_kit.sh and run it. It checks the pinned fingerprints. One failure — the work stops.

5. Two tools must agree. Every time.

Same dice bits → same 24 words in both tools. Same seed → same first addresses in Sparrow and Electrum. The wallet map shows those same addresses. Before signing, both tools show the same destination, amount, change, and fee. If they differ — stop.

STOP. A fingerprint that differs by one character — delete the file. A signature that isn't Good — delete the file. Two tools that disagree — stop.

Two boots

One computer, used twice. The first boot watches the wallet and broadcasts finished payments — it never holds a seed. The second boot is the Tails USB: networking off, exactly one seed, then a full shutdown. The seed lives in memory only. Shutdown wipes it.

What "airgapped" means here

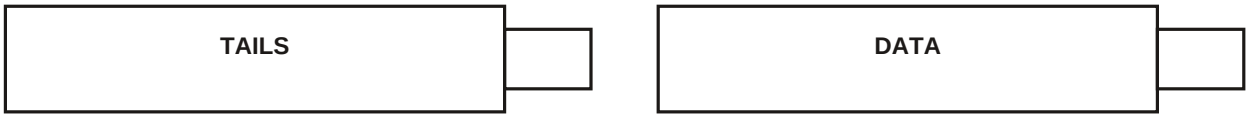
Yes:

- Networking Disabled on the Tails welcome screen, before the desktop.
- Files cross on the data USB — payment files, the map, checked programs.
- The online computer stays watch-only.

Not airgapped:

- Turning networking off after Sparrow opened.
- Unplugging a disk that was online yesterday.
- A photo of a seed. A password manager, printer, or cloud note as backup.

Prepare the two USBs



Tails 7.13 · networking off

checked files only

- Download Tails only from the official site. Check the signature and checksum before writing.
- Write it with the official installer, label it TAILS. Do not create Persistent Storage — skip the passphrase.
- The data USB gets only files that already passed the checks.

Every offline session

1. **Boot the Tails drive. Networking Disabled. Don't unlock Persistent Storage.**
2. **Copy only the file that session needs from the data USB.**

3. Do the work: convert dice, restore one seed, sign one file, or export the map.

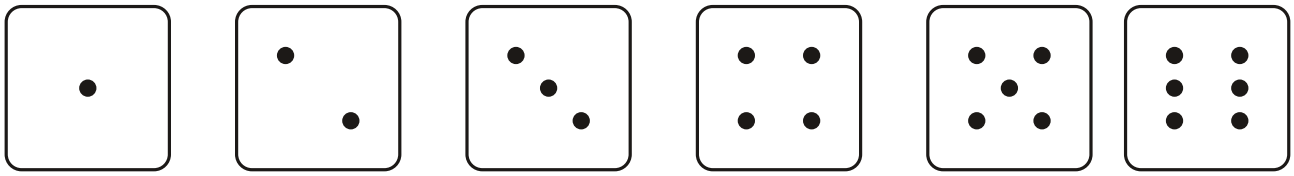
4. Copy the result back to the data USB. Shut down fully — dark screen before the drive comes out.

STOP. Two seeds in one session — never. If networking turns on while a seed is in memory, that session is burned. If Tails offers Persistent Storage, don't create it, don't unlock it.

Stronger (optional): a dedicated old laptop with the Wi-Fi card physically pulled. Ethernet never plugged. The Tails file is still checked, networking still Disabled, data USB still clean — the stick remains the only wire.

Make the three keys

This work starts after Tails has booted with networking set to Disabled, and before any plate is stamped. Five casino dice are required, and every face has to be readable. Faces 1 through 4 are kept. Faces 5 and 6 are rolled again. The work is done three times, for three keys, and the computer is shut down fully between them. An established private setting is required, and there cannot be a camera in the room.



faces 1–4 are kept · faces 5–6 are rolled again

The faces

- 1 is 00. 2 is 01. 3 is 10. 4 is 11.
- 5 and 6 are not bits. Roll those dice again.
- Read the dice left to right, the same way every shake.
- Example: 4, 6, 1, 3, 2 becomes 11, skip the 6, then 00, 10, 01.
- Write only 0 and 1. Roll exactly 256 bits — 128 pairs, about 32 shakes — then stop. More bits do not add security (everything folds down to 256), and extra bits make the two tools below disagree, which stops the work for no reason.

Turn the bits into 24 words

1. Boot Tails. Networking Disabled. Copy the checked converter and the Ian Coleman page from the data USB to the Desktop.

2. Run the converter. Paste the bits. It prints 24 words. Word 24 is the check. Do not invent it. Do not “fix” it.

3. Open the Ian Coleman page in the offline browser. Entropy type: Binary. 24 words. Paste the same bits. The 24 words must match. If they differ, stop. Do not pick the prettier list.

4. Write the 24 words on the grid. Read them back out loud once.

5. Shut Tails down. Wait until the screen is dark. Then start the next key. Seed 2 is a new set of rolls. Seed 3 is a new set of rolls. Never continue a list from the previous boot.

The last word is a check

The 24 words are not 24 free choices. The last word carries a short check so a copying mistake is caught before metal is stamped. At least two of these checks are used. If they disagree, the work stops.

- **Two tools, same bits, same 24 words.** Converter, then the offline Ian Coleman page. Do not pick the prettier list.
- **Let the wallet refuse a bad list.** Sparrow or Electrum, networking off, will open a valid list and reject a broken one. If it will not open, do not change a word to force it.

- **The math, if it needs to be seen.** Twenty-four words are 264 bits. The first 256 are the dice. The last 8 are the check. Only word 24 can change to make that true. It is not chosen.

STOP. If the two word lists differ, stop. Do not pick the nicer list. Do not “fix” word 24 — a fixed word is a destroyed key.

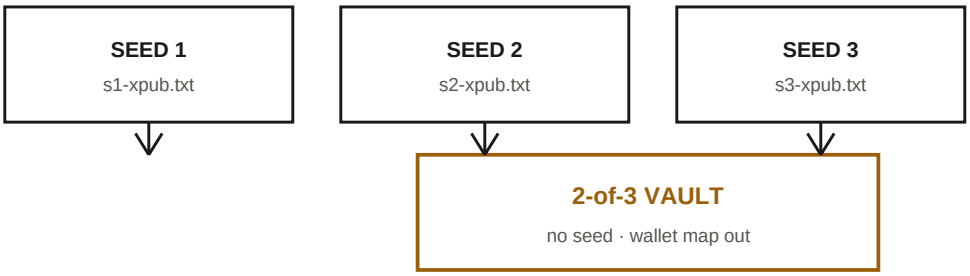
Build the 2-of-3

Three public keys go in. No seed ever touches this build. What comes out is the wallet map — it can watch the money, never spend it.

You will boot Tails four times: once per seed to copy its public key, then once more with no seed to build the vault.

What is required

- The data USB with the checked Sparrow file (sparrowwallet-2.5.5-x86_64.tar.gz)
- The three paper seeds. Only one comes out per boot.



1. Boot Tails. Networking stays off.

The entire sheet happens with networking Disabled. Check it on every boot.

2. Open the terminal. Launch Sparrow.

Type these four lines, one at a time. Wait for the prompt to come back after each. Wait 15–30 seconds. Sparrow opens. Type exactly — spaces count. If a line turns red, stop.

```
$ cd ~/Desktop
$ tar -xvf sparrowwallet-2.5.5-x86_64.tar.gz
$ cd Sparrow
$ ./bin/Sparrow
```

3. New wallet: 2 of 3, native Segwit.

File → New Wallet. Pick Multi Signature, Native Segwit, 2 of 3. The path must read m/48'/0'/0'/2'. Anything else — stop.

4. Keystore 1: Seed 1 in, public key out.

Keystore 1 → New or Imported Software Wallet → type Seed 1's 24 words. Export the xpub. Save it to the data USB as s1-xpub.txt. Close Sparrow. Shut down. Wait for a dark screen.

5. Twice more. One seed per boot.

Boot again for Seed 2 → s2-xpub.txt. Shut down. Boot again for Seed 3 → s3-xpub.txt. Shut down.

6. Fresh boot. No seed. Build the vault.

Boot Tails again — do not type any seed. New wallet → Multi Signature → Native Segwit → 2 of 3 → import the three xpub files. Confirm the path. Export the output descriptor to the data USB. That file is the wallet map.

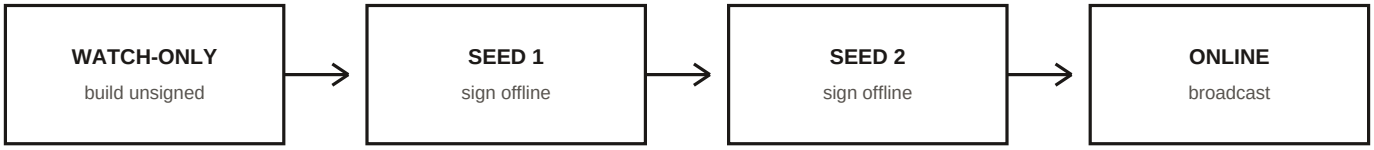
7. Watch-only on the everyday computer.

On the everyday computer, open Sparrow and import the map as a watch-only wallet. It can see the money. It can never spend it.

STOP. The path doesn't read m/48'/0'/0'/2' — stop, don't type a different path. Networking is on at any point — stop. Tempted to type a second seed without shutting down — stop. One seed per boot.

Practice spend

A small amount — enough to pay a fee, not savings. Write the destination, amount, and fee on paper before anything is exported. Read those notes again before each signature.



full shutdown between each signing

1. Build the spend — online, watch-only.

Open watch-only Sparrow. Send tab: destination, amount, fee. Finalize for signing. Export tx-unsigned.psbt to the data USB.

2. Seed 1 signs — Tails, networking off.

Boot Tails, networking disabled. Restore Seed 1 only. Open tx-unsigned.psbt, confirm the fields against the paper, sign. Export tx-partial-s1.psbt. Full shutdown.

3. Seed 2 signs — fresh boot.

Fresh Tails boot. Restore Seed 2 only. Open tx-partial-s1.psbt — confirm the first signature and that the spend is unchanged — sign. Export tx-signed.psbt. Full shutdown.

4. Broadcast — online computer.

Watch-only Sparrow: open the fully signed file, confirm the fields are unchanged, broadcast. Record the txid in the recovery workbook.

Keys on hardware later? Generate the seed on airgapped Tails first, then import the finished 24 words — never let the device roll its own entropy for this stack. Two keys on two brands (Foundation Passport, Blockstream Jade, BitBox02, SeedSigner/Krux — official shops only). Never three of the same box.

STOP. The fields don't match the paper note — stop. Sparrow and Electrum disagree about the file — do not sign. Test with a small amount first. Always.

Metal and the map

A small payment has gone out and come back. Now the keys leave paper for good.

Three titanium plates. One seed per plate. Three separate places. If two plates sleep in the same house, the separation is gone — and the single point of failure is back.

HEIR NEEDS ANY TWO PLATES + THE MAP

The plates

- One seed per plate. Never two.
- StampSeed (recommended): the kit holds 1 plate, buy 2 more. Cryptotag: the kit holds 2, buy 1 more. Pick one method; don't mix them.
- Three places, far enough apart that one fire, flood, or burglary can't take all three.
- No photo, no note, no password manager, no email. Any copy outside the plates is a leak.

How to stamp

1. **Private room, no camera. Plate on a solid block, each letter stamped deep.**
2. **Words numbered 1–24. Label the plate quietly — S1. No names, no “Bitcoin.”**
3. **Read the plate back against the paper. A mis-stamp is a bricked backup.**
4. **Restore from the plate on offline Tails. First addresses must match.**
5. **Only then destroy the paper — and the bit notes with it.**

The home plate

- These 24 words, alone, open a small wallet. Leave a little bitcoin there: a thief who loads only those words may stop at what they find.
- The same words plus a written passphrase open a modest spending wallet. The passphrase lives on paper — never on the plate, never in the same drawer. No passphrase on the 2-of-3 itself.

Where each plate lives

- **Plate A** — Seed 1 — home safe. Never Seed 2 or 3 here.
- **Plate B** — Seed 2 — off-site #1. Not the same address as A or C.
- **Plate C** — Seed 3 — off-site #2. Not the same address as A or B.

Print the output descriptor and keep it with Plate A, plus one copy off-site — never beside all three seeds. Hardware wallets don't replace these plates. Once a year: restore from the plates, receive dust, spend dust, update locations.

Write a letter pointing to the packet and which two locations matter — not the words.

STOP. Paper is destroyed only after the restore check passes. Not before.

The wallet map exists before this work, and it is not left in a drawer as plain text. The map can watch the money. It cannot spend. Anyone who can read it can see the balance, so it is encrypted. The passphrase that opens the file is not a seed, and it is not a plate.

What is required

- The wallet map from the 2-of-3, on the data USB. It is called map.txt only while the work is happening.
- descriptor_obfuscator.py. This tool calls GPG with AES-256. It does not invent a cipher. Its fingerprint is checked in VERIFY.md before it is copied.
- A passphrase that can be remembered and also written down. It is not one of the 24-word lists. It is not stamped on a plate.
- Tails, with networking set to Disabled. This is not done on the everyday computer.

The steps

01. Tails is booted. Networking is set to Disabled. The tool and map.txt are copied to the Desktop. The map is not opened on a phone.
02. Terminal is opened. The tool is run. E is chosen. The wallet map is pasted. The passphrase is entered, and it is not shown on screen.
03. The tool writes map.txt.gpg. That file is copied to two USB drives. map.txt is deleted. The trash is emptied.
04. The passphrase is written on paper. That paper is kept away from the plates and away from the USB drives.
05. On a later offline boot, D is chosen. The map has to open, and the first address has to match. The terminal is then cleared with reset. A plain copy is deleted.

```
cd ~/Desktop
python3 descriptor_obfuscator.py
```

Stop if

- The xpubs are not written on paper just in case.
- A letter code or a homemade cipher is not invented. The tool above is the one that is used.
- The passphrase is not one of the seeds, and it is not stamped on a plate. A different passphrase is chosen, and the work is started over.

If this passphrase is lost, the coins are not gone as long as all three plates survive — three plates rebuild the map exactly. Two plates alone cannot: the third key cannot be derived from the other two. Back up the passphrase paper in two places, away from the plates and USB drives. The path is m/48'/0'/0'/2'. Write that path on the inheritance sheet. It is not a secret.

Next. If I am gone. That page is for a spouse. It does not contain words.

If I am gone

This page is for someone who has **never used Bitcoin**. Read it before touching a plate. One plate cannot move the long-term fund — it needs **any two plates plus the wallet map**.

- ✕ Never type the words into a **phone** or a **website**.
- ✕ Never **photograph** the plates.
- ✕ Never put **both plates** into one device.
- ✕ Never buy a device from a **marketplace** or a link in a message.

HEIR NEEDS ANY TWO PLATES + THE MAP

1. Find two plates in two places.

Leave the third where it is. You only need two.

2. Find the wallet map.

If it asks for a passphrase, that paper is stored **away from the plates**. Decrypt it on a trusted computer — or with someone who can follow the map sheet. The map is not the words.

3. Buy two signing devices — two different makers.

A **Jade** from Blockstream's own shop. A **BitBox** from BitBox's own shop. Not two of the same box.

4. One plate per device.

Plate A into one device. Plate B into the other. **Never both plates in one device.**

5. Load the map on both.

So both devices are looking at the same fund. The path is **m/48'/0'/0'/2'** — that line is not a secret.

6. Send a small test first.

If the test arrives, the fund can move. If the devices disagree, **stop** — don't force it.

The home plate is a decoy. Its words alone open a small wallet — that is not the long-term fund. The fund still needs a second plate. Don't mistake the home balance for the whole savings.

What am I actually making?

Three keys are made at home, with dice, on a computer that is not on the internet. Any two of them can spend. One cannot. The keys are stamped into titanium and kept in three places. A hardware wallet is optional, and it does not create the keys.

I already own a hardware wallet. Do I start there?

No. The device stays in the drawer until the plates exist and a small payment has moved. A hardware signer does not create the key. If a key is later loaded onto a device, a second key goes onto a device from a different maker. All three keys are not put on hardware. The plates are the backup.

The download is newer than the version on the sheet?

Stop. This kit checks Tails 7.13, Sparrow 2.5.5 (sparrowwallet-2.5.5-x86_64.tar.gz), and Electrum 4.8.2. A newer file will not match. That is a stop, not an upgrade. Delete it.

The two word lists do not match?

Stop. Do not pick the list that looks nicer. Do not change word 24 to force a wallet to open. Roll again, or find the copying mistake. A fixed word is a destroyed key.

Can I keep the words in a password manager?

No. A photo, a note, a password manager, or an email is one hacked account away from the whole seed. The words go on metal. The map is encrypted. The passphrase for the map is on paper, away from the plates.

What if the map is lost, or its passphrase is lost?

The coins are not gone if all three plates survive — three plates rebuild the map exactly. Two plates alone cannot rebuild it. The path is 2-of-3, native SegWit, m/48'/0'/0'/2'. Write that path on the inheritance sheet. It is not a secret.

What does my family do?

Give them the If I am gone sheet. They need two plates and the map. They buy a Jade and a BitBox from the makers' own shops. One plate goes in each device. They do not boot Tails. They do not type the words into a phone.

Next. The last sheet is only if someone challenges the method. It is not required to finish the setup.

These are the three questions people use to poke a hole in the method. The answer is only what is true. A step that is not on the sheets is not added.

The words were typed on a laptop.

Yes. On Tails, with networking off, one seed, then a full shutdown. Tails runs from memory and forgets the session. That does not stop a hardware keylogger. One captured seed still cannot spend. The fund needs two keys. Two seeds are never typed on the same boot.

Why not start with a SeedSigner?

SeedSigner is a signer that can be added later. It is not how these keys are made. The randomness is dice. The computer is ordinary, and it is offline. The inheritance sheet is not a SeedSigner guide. It is the letter for heirs: one plate in a Jade, one plate in a BitBox.

If the map passphrase is lost, are the coins gone?

No. That passphrase hides the map. It does not spend. Three plates rebuild the map exactly if the path is known: 2-of-3, native SegWit, m/48'/0'/0'/2'. Two plates alone cannot. Keep that path outside the encrypted file. A homemade cipher is not used.